TimescaleDB:
Gerenciamento de Séries Temporais no PostgreSQL

Kelwin Efrain Bagnhuk da Silva e Uriel Pacheco de Souza

Banco de Dados II

Professor Fabiano Baldo

Joinville, 2026

UDESC
UNIVERSIDADE DO ESTADO DE SANTA CATARINA

Sumário

1 **Introdução:**
Séries temporais, padrões de acesso e por que tabelas comuns degradam

2 **Definições:**
TimescaleDB e instalação, hypertable, chunks, EXPLAIN, aggregate e compressão

3 **Demonstração Prática:**
Hypertable e inserção, chunks com EXPLAIN ANALYZE, aggregate e compressão

4 **Aplicação: caso de uso:**
Sensores IoT: tabela comum vs. hypertable, com comparação de desempenho

5 **Conclusão**
Limitações, quando usar, conclusão.

6 **Referências**
Fontes utilizadas

Introdução

Objetivo

Armazenar e consultar dados que chegam continuamente e em grande quantidade, como em sensores IoT, métricas de sistemas, cotações financeiras e logs de eventos.
Cada registro precisa ter um **carimbo de tempo**, e o que interessa é **como os valores evoluem**.

Como fazer isso com bom desempenho e uso de disco controlado?

Limitações do PostgreSQL puro

Inserção contínua deixa o índice pesado

Cada inserção também atualiza o índice. Conforme a tabela cresce, o índice cresce junto e fica mais caro de manter.

Apagar dados antigos é caro

No PostgreSQL, o DELETE só marca as linhas como mortas. O espaço só é reaproveitado após o VACUUM, e até lá a tabela fica inchada e mais lenta.

Consulta por período lê uma tabela única

Todos os dados ficam em uma só tabela. Sem índice em "tempo" a consulta varre tudo. Com índice, o custo do índice e da tabela cresce com o volume.

Agregações repetem o mesmo cálculo

Uma média por hora com GROUP BY lê todas as linhas do período a cada consulta. Sem resumo pré-calculado, o custo cresce junto com a tabela.

Referências: 1, 2, 3, 4

Definições: Séries Temporais

Definição

Dados de séries temporais são uma sequência de pontos de dados coletados em intervalos de tempo que rastreiam mudanças ao longo do tempo, fornecendo informações sobre tendências, padrões e relações.

- Alta taxa de inserção (dados novos chegam o tempo todo);
- Consultas por janela de tempo ("última hora", "últimos 7 dias");
- Agregações por intervalo (média por hora).

Exemplo: Sensores IoT

tempo	sensor_id	temp. (°C)	umidade (%)
10:01:00	1	24,1	61
10:02:00	2	23,7	58
10:02:00	1	24,2	61

Cada sensor envia uma leitura por minuto;

Consultas típicas seriam: "média da última hora" ou "máximo por dia"

Referências: 6

Definições: TimescaleDB

TimescaleDB

O **TimescaleDB** é um banco de dados de séries temporais construído como uma extensão do PostgreSQL. Ele aumenta o desempenho do PostgreSQL, transformando-o efetivamente em um banco de dados de séries temporais

Ele fornece os mesmos recursos de consulta SQL do PostgreSQL, com otimizações adicionais específicas para séries temporais. Ele pode lidar com altas cargas de gravação e consulta típicas de dados de séries temporais, oferecendo recursos como particionamento automático, compressão e análises em tempo real

Exemplo: Instalação com Docker

```
docker run -d --name timescaledb \
-p 5432:5432 -e
POSTGRES_PASSWORD=senha \
timescale/timescaledb:latest-pg18
```

No pgAdmin4: host "*localhost*" e a porta à esquerda do -p (5432). A versão do PostgreSQL é definida pela tag (-pg18). Se já houver um PostgreSQL local na 5432, use -p 5433:5432.

```
CREATE EXTENSION IF NOT EXISTS
timescaledb;
SELECT extversion FROM pg_extension
WHERE extname = 'timescaledb';
```

Para verificar a versão:
SELECT version();

Referências: 5, 6, 7

Definições: TimescaleDB

Exemplo: Instalação Linux

Antes de realizar a instalação, é necessário adicionar o repositório oficial da Timescale. Após isso, rodar no Terminal os comandos e reiniciar o PostgreSQL:

```
sudo apt install -y
timescaledb-2-postgresql-18

sudo timescaledb-tune --quiet --yes
```

```
CREATE EXTENSION IF NOT EXISTS
timescaledb;
SELECT extversion FROM pg_extension
WHERE extname = 'timescaledb';
```

Exemplo: Instalação Windows

- **Instalar o PostgreSQL nativo no Windows.**
- **Baixar e executar o instalador oficial .zip ou executável (setup.exe) fornecido pela Timescale para Windows.**
- **Reinicie o PostgreSQL.**

```
CREATE EXTENSION IF NOT EXISTS
timescaledb;
SELECT extversion FROM pg_extension
WHERE extname = 'timescaledb';
```

Ativação Padrão para todos métodos

Referências: 5

Definições: TimescaleDB

Exemplo: Instalação Linux

Antes de realizar a instalação, é necessário adicionar o repositório oficial da Timescale. Após isso, rodar no Terminal os comandos e reiniciar o PostgreSQL:

```
sudo apt install -y
timescaledb-2-postgresql-18

sudo timescaledb-tune --quiet --yes
```

```
CREATE EXTENSION IF NOT EXISTS
timescaledb;
SELECT extversion FROM pg_extension
WHERE extname = 'timescaledb';
```

Exemplo: Instalação Windows

- **Instalar o PostgreSQL nativo no Windows.**
- **Baixar e executar o instalador oficial (.zip) ou executável (setup.exe) fornecido pela Timescale para Windows.**
- **Reinicie o PostgreSQL.**

```
CREATE EXTENSION IF NOT EXISTS
timescaledb;
SELECT extversion FROM pg_extension
WHERE extname = 'timescaledb';
```

Para verificar se funcionou:

```
SELECT extversion FROM pg_extension WHERE extname = 'timescaledb';
```

Referências: 5

Definições: Hypertable e Chunks

As **hypertables são tabelas do PostgreSQL** que ajudam a melhorar o desempenho de inserção e consulta **particionando automaticamente seus dados por tempo**

Como funciona

O TimescaleDB divide seus **dados em chunks** que armazenam dados de um intervalo de tempo específico [8]:

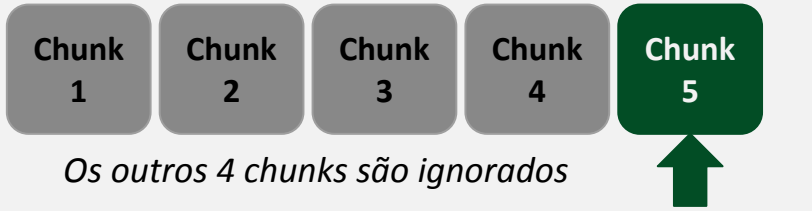
- create_hypertable converte a tabela, com by_range('time')
- chunks criados automaticamente (padrão: 7 dias)
- cada chunk tem índices próprios, menores
- chunk exclusion: consulta com filtro de tempo lê só os chunks da janela

Comparação

	PostgreSQL	TimescaleDB
Partições	PARTITION BY RANGE, criar manualmente	Chunks automáticos
Consulta	A performance depende da B-TREE	Descarta chunks fora do intervalo
Retenção	DELETE + VACUUM	Descarte por chunks inteiros

(Tabela comum)

sensor_data - 30 dias - 432.000 linhas



Referências: 1, 2, 3, 8

Definições: EXPLAIN ANALYZE

O EXPLAIN é a estimativa do banco, já o EXPLAIN ANALYZE executa de fato e cronometra o tempo real, comprovando a leitura exata dos chunks.

Como funciona

EXPLAIN:
Mostra apenas a estimativa do planejador (custo e linhas esperadas) sem rodar o comando no banco de dados.

EXPLAIN ANALYZE:
Executa a consulta e mostra o plano com o tempo real e as linhas processadas em cada etapa.

Código

```
EXPLAIN ANALYZE
SELECT avg(temperatura)
FROM sensor_data
WHERE time >= now()
- interval '1 hour';
```

EXPLAIN: só estimativa, não executa
EXPLAIN ANALYZE: executa e mede

Referências: 10

Definições: EXPLAIN ANALYZE

```
EXPLAIN ANALYZE
SELECT avg(temperatura) FROM sensor_data
WHERE time >= now() - interval '1 hour';
```

Custom Scan (ChunkAppend): Leitura particionada nativa do TimescaleDB;
Bitmap Index Scan / Bitmap Heap Scan: usa o índice de tempo para localizar as linhas (não varre a tabela toda);
Chunks listados: só o _hyper_1_5_chunk aparece, houve chunk exclusion;
Execution Time: Tempo real e cronometrado no hardware (0.254 ms).

	QUERY PLAN text
1	Aggregate (cost=378.30..378.31 rows=1 width=8) (actual time=0.201..0.201 rows=1 loops=1)
2	-> Custom Scan (ChunkAppend) on sensor_data (cost=7.53..376.80 rows=599 width=8) (actual time=0.028..0.160 rows=570 loops=1)
3	Chunks excluded during startup: 0
4	-> Bitmap Heap Scan on _hyper_1_5_chunk (cost=7.53..376.80 rows=599 width=8) (actual time=0.027..0.116 rows=570 loops=1)
5	Recheck Cond: ("time" >= (now() - '01:00:00'::interval))
6	Heap Blocks: exact=13
7	-> Bitmap Index Scan on _hyper_1_5_chunk_sensor_data_time_idx (cost=0.00..7.39 rows=599 width=0) (actual time=0.019..0.019 rows=570 lo...
8	Index Cond: ("time" >= (now() - '01:00:00'::interval))
9	Planning Time: 0.310 ms
10	Execution Time: 0.254 ms

Fonte: Elaborado pelos autores, 2026.

Definições: continuous aggregate

Definição

As agregações contínuas (continuous aggregates) são um tipo de hypertable que é atualizada automaticamente em segundo plano à medida que novos dados são adicionados ou dados antigos são modificados (e de acordo com a política de *refresh*).

Atenção: a agregação em tempo real vem desativada por padrão nas versões recentes [4].

Exemplo

sensor_data (bruto)

tempo	sensor_id	temp. (ºC)
10:00:00	1	24,6
10:00:00	2	26,1
...	...	...
12:00:00	10	24,2

sensor_hourly

tempo	temp. média	leituras
10:00:00	25	600
11:00:00	25	600
12:00:00	25	600

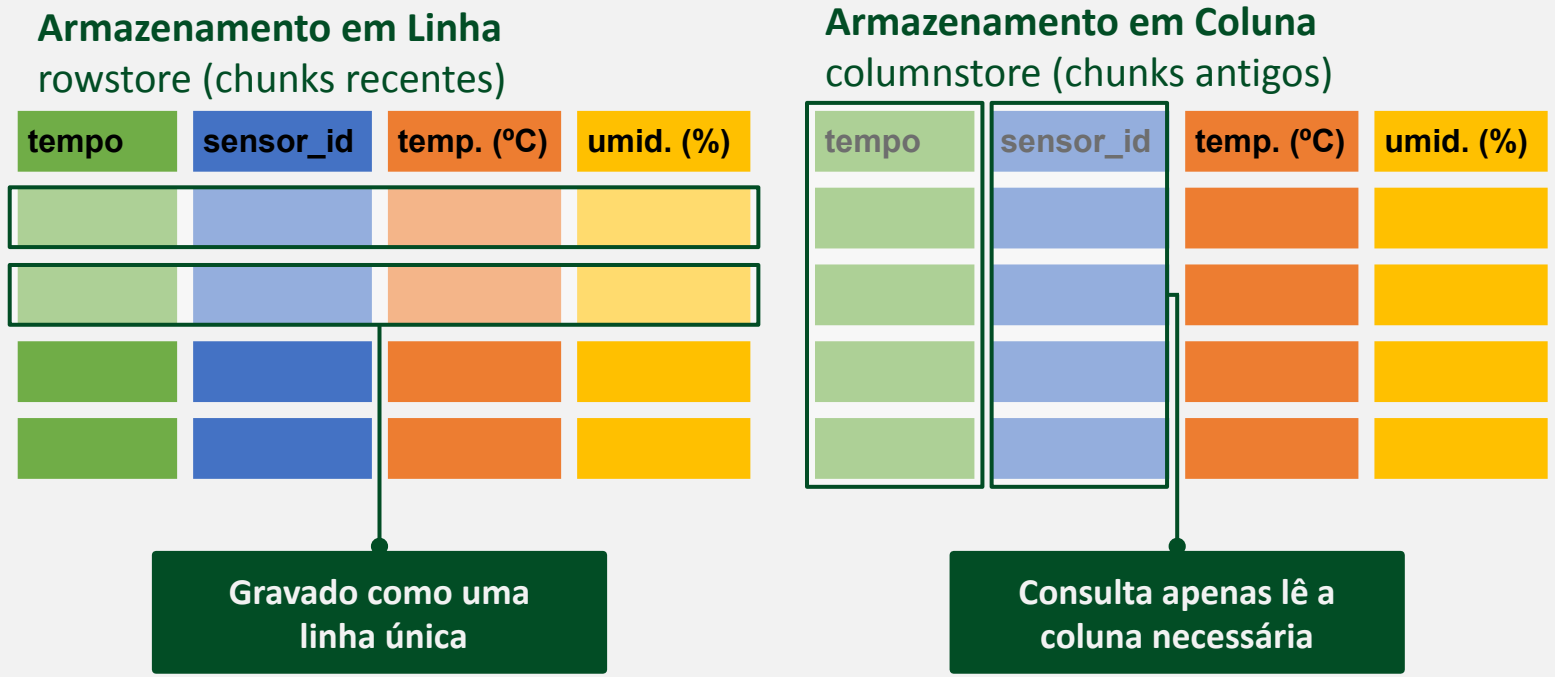
Referências: 4, 11

Definições: compressão (columnstore)

A compressão converte chunks antigos do armazenamento em linhas (rowstore) para colunas (columnstore), reduzindo o espaço em disco e acelerando consultas analíticas sobre períodos longos [9]

Como funciona

Os chunks recentes continuam em linhas para inserções rápidas.
Os antigos, que quase não mudam, são comprimidos. O segmentby agrupa por sensor_id e o orderby ordena por tempo.



A documentação oficial cita reduções de até 98% [9]

Referências: 9

Demonstração 1

Hypertable e inserção

1. Habilitar a extensão
2. Criar a tabela e converter em hypertable
3. Inserir dados com generate_series
4. Listar os chunks criados

Código

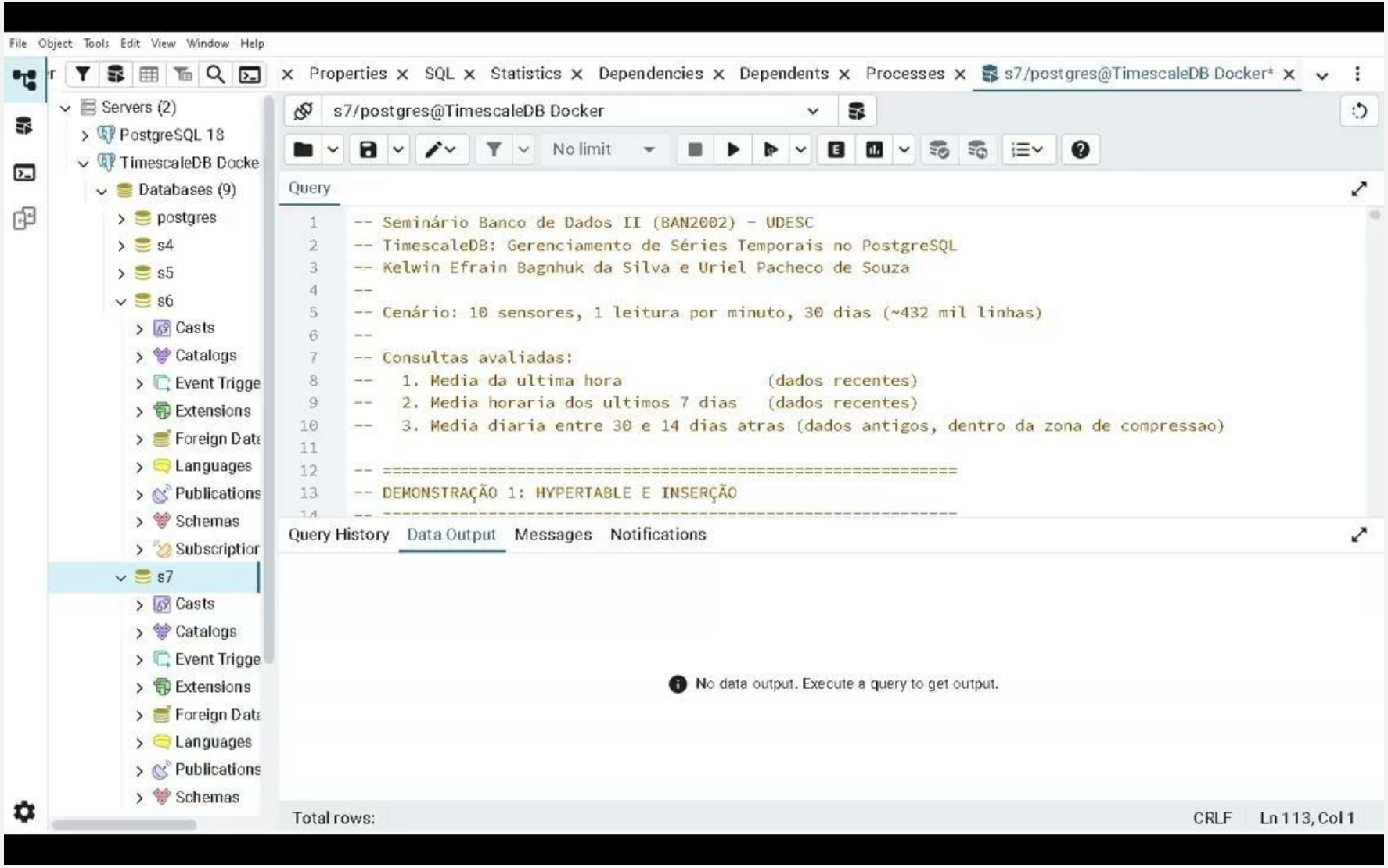
```
CREATE TABLE sensor_data (
    time        TIMESTAMPTZ  NOT NULL,
    sensor_id   INTEGER      NOT NULL,
    temperatura DOUBLE PRECISION,
    umidade     DOUBLE PRECISION
);
SELECT create_hypertable('sensor_data',
by_range('time'));
```

```
INSERT INTO sensor_data (time, sensor_id, temperatura, umidade)
SELECT
    t,
    s,
    20 + random() * 10,
    50 + random() * 30
FROM generate_series(
     now() - interval '30 days', --início
     now(),                      --fim
     interval '1 minute') AS t,  --cada minuto
     generate_series(1, 10) AS s;--cada sensor
```

Resultado: 432.010 linhas em 5 chunks de 7 dias*

**30 dias * 24h * 60min = 43200 intervalos + 1 = 43201 instantes*
*43201 * 10 sensores = 432010 registros*

Demonstração 1



Demonstração 2

Desempenho com EXPLAIN ANALYZE

1. Copiar os dados para uma tabela comum
2. EXPLAIN ANALYZE sem índice (Seq Scan)
3. Criar índice e repetir
4. Repetir na hypertable e comparar

Código

```
-- Consulta 1 na hypertable
EXPLAIN ANALYZE
SELECT avg(temperatura)
FROM sensor_data
WHERE time >= now() - interval '1 hour';
```

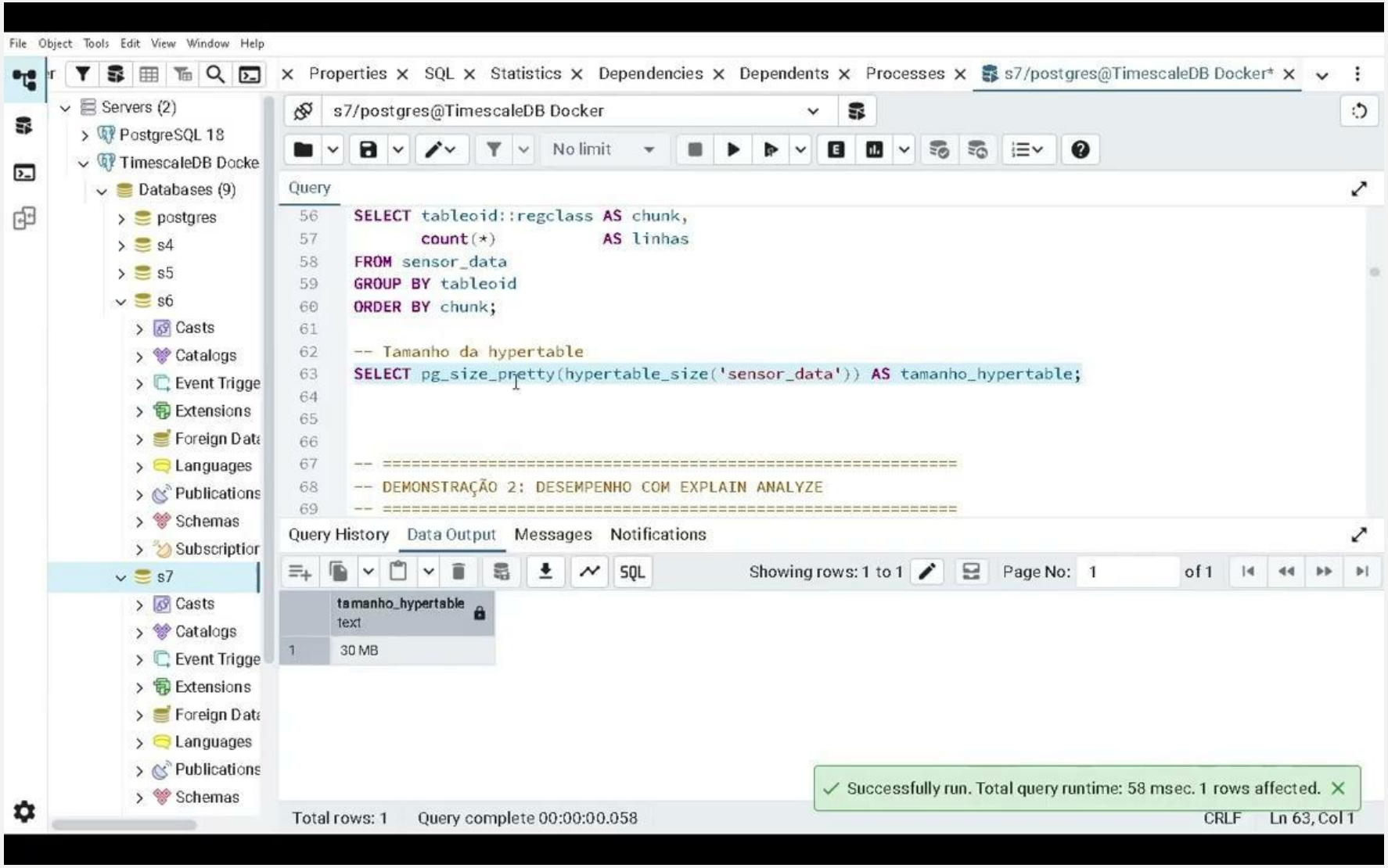
```
-- 5. Tamanho (dados, índices e total)
-- Hypertable
SELECT pg_size_pretty(table_bytes) AS dados,
       pg_size_pretty(index_bytes) AS indices,
       pg_size_pretty(total_bytes) AS total
FROM hypertable_detailed_size('sensor_data');

-- Tabela comum
SELECT pg_size_pretty(pg_table_size('sensor_data_plain'))          AS dados,
       pg_size_pretty(pg_indexes_size('sensor_data_plain'))        AS indices,
       pg_size_pretty(pg_total_relation_size('sensor_data_plain')) AS total;
```

Resultado:
*Hypertable 25MB + 5MB = 30MB**
*Tabela Comum 25MB + 4MB = 29MB**

**Hypertable possui um conjunto de índices para cada chunk*

Demonstração 2



Demonstração 3

Aggregate e compressão

1. Criar a continuous aggregate e a política
2. Consultar o agregado vs. a tabela bruta
3. Medir o tamanho, comprimir e medir de novo

Código

```
CREATE MATERIALIZED VIEW sensor_hourly
WITH (timescaledb.continuous) AS
SELECT time_bucket('1 hour', time) AS hora,
       avg(temperatura)          AS temp_media,
       avg(umidade)              AS umid_media,
       count(*)                  AS leituras
FROM sensor_data
GROUP BY hora
WITH NO DATA; -- 721 linhas (24 * 30 intervalos)
```

```
ALTER TABLE sensor_data SET (
    timescaledb.enable_columnstore = true,
    timescaledb.segmentby = 'sensor_id',
    timescaledb.orderby   = 'time DESC'
);
```

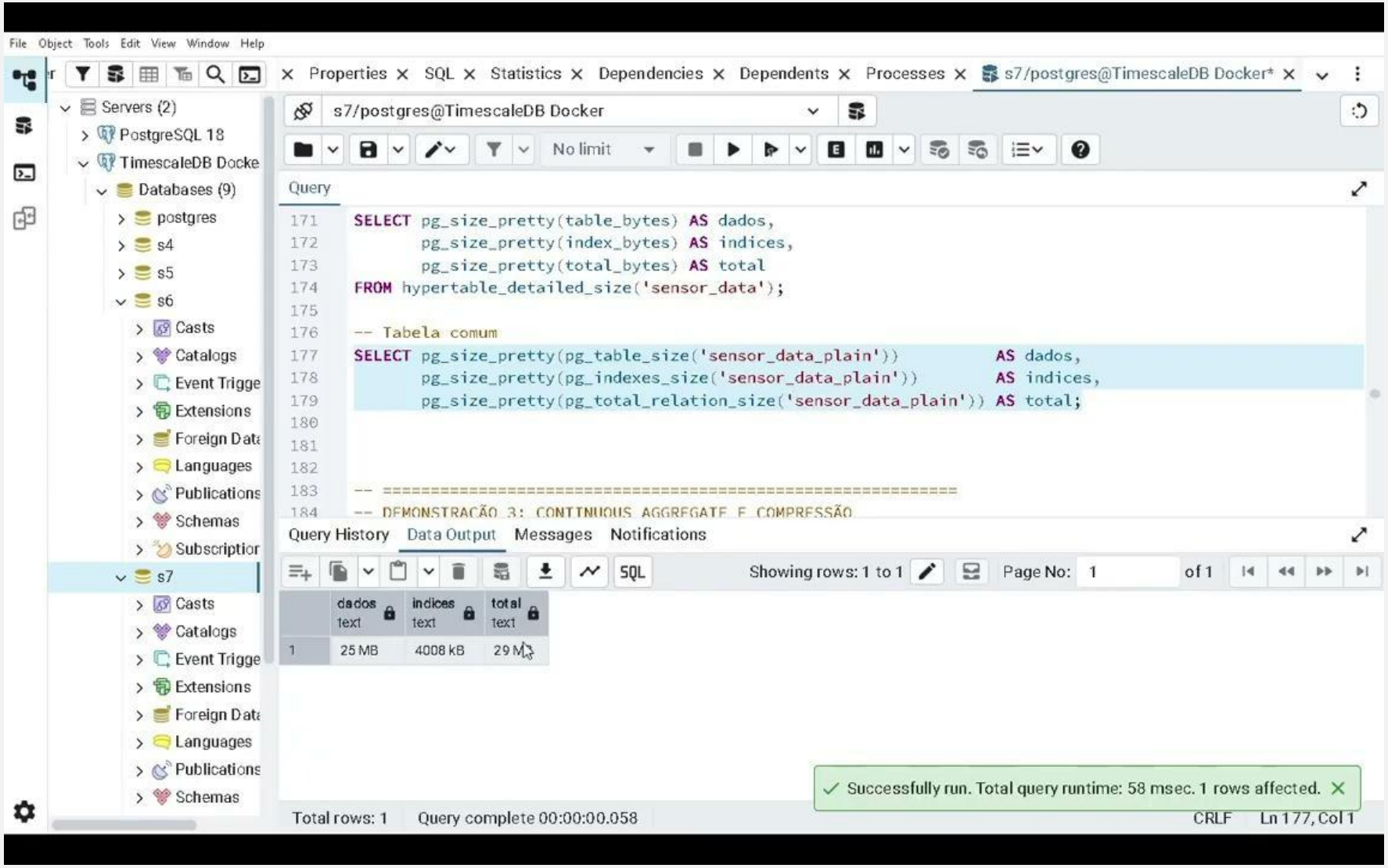
Ativa o armazenamento em colunas e define o agrupamento por sensor e a ordenação por tempo

```
-- Política: comprime automaticamente os chunks com mais de 7 dias
CALL add_columnstore_policy('sensor_data', after => INTERVAL '7 days');

-- Comprime agora os chunks antigos (a política roda em segundo plano)
SELECT compress_chunk(c, if_not_compressed => true)
FROM show_chunks('sensor_data', older_than => INTERVAL '7 days') c;
```

- *Agenda uma rotina para comprimir automaticamente qualquer chunk que tenha mais de 7 dias de dados.*
- *Força a compressão imediata de todos os chunks antigos no momento da execução do script*

Demonstração 3



Aplicação: caso de uso IoT

Cenário

Monitoramento de 10 sensores de temperatura e umidade, com 1 leitura por minuto, durante 30 dias.

Mesmo dataset em uma tabela comum e em uma hypertable.

Consultas avaliadas

1. Média da última hora
2. Média horária dos últimos 7 dias
3. Média diária entre 30 e 14 dias atrás

Comparação de Desempenho

Métrica	Tabela Comum sem Índice	Tabela Comum com Índice	Hypertable	Aggregate	Com compressão
Tamanho	25MB	29MB	30 MB	128KB	16MB
Consulta 1	17ms	0,17ms	0,26ms	0,1ms	0,25ms
Consulta 2	68ms	27ms	34ms	0,1ms	34ms
Consulta 3	138ms	59ms	79ms	0,3ms	16ms

Execution Time, mediana de N execuções

Conclusão

Quando USAR?

- Dados com carimbo de tempo, inseridos continuamente (IoT, métricas, finanças);
- Consultas por janela de tempo e agregações por hora ou dia;
- Volume que cresce muito e exige controle de espaço e retenção.

Quando NÃO USAR?

- Poucos dados ou sem eixo temporal. PostgreSQL comum com índice basta.
- Quando não há necessidade de agregações recorrentes, compressão ou retenção por tempo
- Restrição de chaves: em hypertables, PRIMARY KEY e UNIQUE precisam incluir a coluna time.

Pontos negativos dos experimentos:

- Em 432 mil linhas, a hypertable sozinha foi 20 a 50% mais lenta que a tabela com índice e ocupa ligeiramente mais espaço;
- A compressão não acelera consultas em dados recentes (últimos 7 dias: 34 ms vs 34 ms), pois atua apenas em chunks antigos.

Pontos positivos dos experimentos:

- Continuous aggregate: 34 ms para 0,1 ms na média horária de 7 dias (e 79 ms para 0,3 ms na média diária)
- Compressão: reduziu a hypertable de 30 MB para 16 MB e acelerou a Consulta 3 de 79 ms para 16 ms;
- Mesmo SQL e mesmas ferramentas do PostgreSQL.

Referências

- [1] POSTGRESQL. **Index types**. [S. l.]: PostgreSQL Global Development Group, 2026. Disponível em: https://www.postgresql.org/docs/current/indexes-types.html. Acesso em: 17 out. 2026.
- [2] POSTGRESQL. **Routine vacuuming**. [S. l.]: PostgreSQL Global Development Group, 2026. Disponível em: https://www.postgresql.org/docs/current/routine-vacuuming.html. Acesso em: 17 out. 2026.
- [3] POSTGRESQL. **Table partitioning**. [S. l.]: PostgreSQL Global Development Group, 2026. Disponível em: https://www.postgresql.org/docs/current/ddl-partitioning.html. Acesso em: 17 out. 2026.
- [4] TIGER DATA. **Understand continuous aggregates**. [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/learn/continuous-aggregates. Acesso em: 18 out. 2026.
- [5] TIGER DATA. **Install TimescaleDB**. [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/get-started/choose-your-path/install-timescaledb#tab=linux. Acesso em: 17 out. 2026.
- [6] TIGER DATA. **What is a time series and how is it used?** [S. l.]: Tiger Data, 7 ago. 2024. Disponível em: https://www.tigerdata.com/blog/time-series-introduction. Acesso em: 17 out. 2026
- [7] TIGER DATA. **Tiger Cloud essentials.** [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/learn/tiger-cloud/tiger-cloud-essentials. Acesso em: 17 out. 2026.

UDESC
UNIVERSIDADE
DO ESTADO DE
SANTA CATARINA

Referências

- [8] TIGER DATA. **Hypertables and chunks**. [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/reference/timescaledb/hypertables. Acesso em: 17 out. 2026.
- [9] TIGER DATA. **Understand hypercore**. [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/learn/columnar-storage/understand-hypercore. Acesso em: 17 out. 2026.
- [10] POSTGRESQL. **Using EXPLAIN**. [S. l.]: PostgreSQL Global Development Group, 2026. Disponível em: https://www.postgresql.org/docs/current/using-explain.html. Acesso em: 17 out. 2026.
- [11] TIGER DATA. **Real-time aggregates**. [S. l.]: Tiger Data, 2026. Disponível em: https://www.tigerdata.com/docs/learn/continuous-aggregates/real-time-aggregates. Acesso em: 18 out. 2026.